



crategraph cheat sheet

Researcher-friendly graph exploration for RO-Crate data. Every filter and transform returns a new Graph, so they chain.

■ General ■ Edges ■ Nodes ■ Operations
Inspired by Franz Diebold's Polars cheat sheet (MIT).

THE DATA MODEL

Layer	List	Filter by
Entities (nodes)	crate.entities	where(), a property value
Relationships (edges)	crate.relationships	select(), a type or structure

where() filters by a node property. select() filters by an edge type.

LOAD A GRAPH

```
from crategraph import Crate

crate = Crate("crates/UMPC") # dir with ro-
                             # crate-metadata.json
crate # Graph(4270 entities, 12601
relationships)

Crate(p1, p2) # several
              # crates at once
Crate(path, include_root=True) # keep root
Dataset node
```

OVERVIEW & ANALYSE

```
crate.summary() # types + relationship
counts
crate.profile() # density, components
crate.coverage() # rel coverage by type
crate.glimpse() # type-level visual
len(crate) # 4270
crate.most_connected(n=3)
```

RELATIONSHIPS (EDGES)

```
list(crate.relationship_types)
crate.relationship_counts("type")
crate.relationship_records(
    columns=["source", "target", "type"])

crate.select(relationship_types="Primary")
crate.select(relationship_types=["Previous",
"Related"])
```

EDGES: PATTERN & DERIVE

```
crate.pattern(via="preparedBy") # by edge type

crate.annotate_relationships(
    pair=lambda r: f"{r.source.type} →
{r.target.type}"
).relationship_counts("pair")

crate.collapse_edges() # merge
parallel edges
```

SUBSET THE GRAPH

```
crate.select(entity_types=["Person"])
crate.select(time_range=(1945, 1960))
crate.select(min_connections=20)
crate.exclude(relationship_types="Related")
crate.drop(preparer, property="preparedBy")
crate.subtract(other)
```

SEARCH (FUZZY)

```
crate.search("Elisabet") # default
score ≥ 80
crate.search("Elisabeth",
    threshold=90, top_n=3)
```

Returns a subgraph, so it chains with where and expand.

ENTITIES (NODES)

```
crate.entities # all nodes
crate.files # file entities
list(crate.types) # all @type
names
e = crate.where(name="...").entities[0]
crate.get(e.id) # by id
crate.entity_view(e.id) # live
EntityView
```

PROPERTIES & COUNTS

```
e.properties # the data dict
sorted({k for e in crate.entities
    for k in e.properties}) # all keys
crate.entity_counts("function") # value counts
crate.entity_records(
    columns=["id", "label", "type"])
```

FILTER ENTITIES

```
crate.where(function="Lawyer") # by value
crate.where(startDate=(1870, 1900)) #
inclusive range

crate.annotate_entities(
    state=lambda e:
e.related("birthState").first("name")
).where(state="Victoria")
```

TRANSFORM

```
crate.convert_dates() # strings to
ISO/year
crate.convert_dates(start="startDate",
end="endDate")
crate.merge_nodes(by="type") # type-level
view
crate.detect_communities(seed=42) # adds
"community"
crate.where(function="Lawyer").expand(depth=1)
crate.query("(a:Person)-[:birthPlace]→
(b:Place)")
```

FILE TEXT & SEMANTIC

```
len(crate.files)
next(crate.text_records(include_properties=
["name"]))

crate.build_semantic_index(store_path="idx.db")
crate.search("scholarships for women",
    mode="semantic",
    store_path="idx.db")
next(crate.chunk_records(" ... ",
    store_path="idx.db", k=1))
```

VISUALISE

```
crate.visualise(colour_by="type",
    filepath="net.html") # 2d
sigma (default)
crate.visualise(renderer="3d",
    filepath="3d.html")
crate.visualise(colour_by="community",
    size_by="year",
    filepath="styled.html")
crate.gallery(columns=4,
    filepath="gallery.html")
```

renderer: 2d, 3d, svg, pyvis.

EXPORT

```
crate.write("crate.graphml", format="graphml")
# Gephi/Cytoscape
crate.to_networkx()           # NetworkX
MultiDiGraph
import pandas as pd
pd.DataFrame(crate.entity_records())
```